

Web-Based Vehicle Auction Platform

System Manual

Appendix A — Installation, Compilation, and Execution Guide

Stack: Node.js · Express · PostgreSQL · React 19 · Vite · Tailwind CSS v4

This manual documents how to install, configure, run, and extend the submitted codebase. It does not describe building the project from scratch — the code already exists in Backend/ and Frontend/.

1. Repository Layout

Backend/

src/{config,db,middleware,repositories,services,controllers,routes}

migrations/ (001–009, numbered SQL files)

scripts/seed-admins.js

Frontend/

src/{api,components,context,pages,router}

2. Prerequisites

Tool	Version	Source
Node.js	20 LTS	https://nodejs.org
PostgreSQL	16	https://www.postgresql.org
Git	any recent	https://git-scm.com

3. Installation

```
git clone <repository-url> vehicle-auction
cd vehicle-auction/Backend && npm install
cd ../Frontend && npm install
```

Backend dependencies (reusable code, all from the public npm registry)

Package	Purpose	Location
express	HTTP server/routing	https://npmjs.com/package/express
pg	PostgreSQL client/pool	https://npmjs.com/package/pg
bcrypt	Password hashing	https://npmjs.com/package/bcrypt
jsonwebtoken	JWT auth	https://npmjs.com/package/jsonwebtoken
joi	Request validation	https://npmjs.com/package/joi
morgan	HTTP logging	https://npmjs.com/package/morgan
node-cron	Auction status scheduler	https://npmjs.com/package/node-cron
dotenv	Env var loading	https://npmjs.com/package/dotenv

Package	Purpose	Location
nodemon (dev)	Auto-restart	https://npmjs.com/package/nodemon

Frontend dependencies

Package	Purpose	Location
react, react-dom	UI framework	https://npmjs.com/package/react
react-router-dom	Client routing	https://npmjs.com/package/react-router-dom
axios	HTTP client	https://npmjs.com/package/axios
tailwindcss, @tailwindcss/vite, @tailwindcss/forms	Styling	https://tailwindcss.com
vite, @vitejs/plugin-react	Build tool	https://vitejs.dev

External service (not npm-hosted)

Cloudinary (image CDN, free tier) (cloudinary.com) used for unsigned direct browser-to-CDN image upload. No image binaries are ever stored in the app or database.

4. Environment Variables

Backend/.env (never committed):

```
PORT=3000
DATABASE_URL=postgresql://user:password@host:5432/vehicle_auction
JWT_SECRET=<random string, 32+ chars>
JWT_EXPIRES_IN=1h
BCRYPT_ROUNDS=10
NODE_ENV=development
```

Frontend/.env:

```
VITE_CLOUDINARY_CLOUD_NAME=<your cloud name>
VITE_CLOUDINARY_UPLOAD_PRESET=<your unsigned preset>
```

5. Database Setup

```
createdb vehicle_auction
for f in Backend/migrations/*.sql; do psql -d vehicle_auction -f "$f"; done
node Backend/scripts/seed-admins.js # creates first admin account
```

Migrations are numbered 001–009 and must be applied in order; never edit an applied migration — add a new one instead. See Chapter 3 (Design) of the main dissertation for the full rationale behind each migration.

6. Running the Application

npm run dev # development, auto-restart
npm start # production

Frontend (from Frontend/)
npm run dev # development server, proxies /api to localhost:3000
npm run build # production build → Frontend/dist

Health check: GET http://localhost:3000/api/health → 200 OK.

7. Compilation Notes

- Backend is plain ECMAScript-module JavaScript — no compile step; "type": "module" must remain in Backend/package.json.
- Frontend is compiled/bundled by Vite via npm run build, producing static assets in Frontend/dist/.

8. API Route Map

Method	Path	Auth	Description
POST	/api/auth/register	None	Register buyer
POST	/api/auth/login	None	Login
GET	/api/users/me	Auth	Own profile
PATCH	/api/users/:id/status	Admin	Verify/reject buyer
GET/POST/PUT/DELETE	/api/vehicles[:id]	Mixed	Catalogue CRUD
GET/POST/PUT/DELETE	/api/auctions[:id]	Mixed	Auction CRUD
GET	/api/auctions/won/me	Auth	Must be declared before /:id
POST	/api/auctions/:id/close	Admin	Close + auto-assign winner
POST	/api/auctions/:id/winner	Admin	Manual winner override
GET/POST	/api/auctions/:id/bids	Public/Verified buyer	Bid history / place bid

9. Actual Deployment (as delivered)

The delivered system is hosted on:

- Backend + PostgreSQL: Railway (managed hosting, automated build from Git)
- Frontend: Vercel (static hosting + CDN)

Steps:

- Push Backend/ and Frontend/ to Git.
- On Railway: create a Postgres instance, deploy Backend/ as a service, set the environment variables from §4, run the migrations and seed script via Railway's shell.

- On Vercel: import Frontend/, set the VITE_* env vars, deploy. Vercel serves the SPA with automatic fallback routing (no manual Nginx try_files config needed).
- Point the frontend's API base URL at the Railway backend's public URL.
- **A self-hosted Ubuntu/Nginx/PM2 deployment is also technically possible with this codebase, but is not how the delivered system is run, and is not documented here to avoid conflicting with the actual production setup.**

10. Known Limitations for Future Extension

Anyone extending this codebase should be aware the following requirements were descoped and are not implemented:

- Buyer registration does not currently capture a contact/phone number (Joi schema and DB insert only cover name/email/password).
- No email or in-app notification system (verification status, outbid alerts) exists.
- No batch/CSV vehicle upload.
- Payment processing and shipping/logistics tracking are out of scope.

11. Common Pitfalls

- Missing .js extensions on ESM imports will fail at runtime.
- SQL must always be parameterized (\$1, \$2...) — never string-concatenated.
- /api/auctions/won/me must be registered before /api/auctions/:id.
- CountdownTimer and the 15s auction-detail polling both require useEffect interval cleanup.
- The JWT stored client-side is stale until re-login; /api/users/me is the source of truth for verification status.